



车位锁系统流程图文档

本文档使用 Mermaid 语法绘制，可在支持 Mermaid 的编辑器（VS Code、Typora、GitHub 等）中直接渲染为图形。

1. 总体系统架构图

```
flowchart LR
    subgraph 现场层 [现场层 / Field Layer]
        S1["从机1<br/>stm32f103_ot_sensor"] -->|RS485 半双工| GW
    end
    subgraph 网关层 [网关层 / Gateway Layer]
        GW["网关主机<br/>stm32f103_ot_gateway"]
    end
    GW -->|USART1<br/>115200 ASCII| SCR["串口屏<br/>Nextion HMI"]
    GW -->|USART2<br/>115200 ASCII| ESP["ESP8266<br/>WiFi透传模块"]
    ESP -->|MQTT over WiFi| CLOUD["云端服务器<br/>FastAPI + WebUI"]
    SCR -. "触摸控制指令" .-> GW
    CLOUD -. "HTTP / WebSocket" .-> BROWSER["浏览器前端"]
```

说明

- **从机**：负责传感器采集（DHT11 温湿度、火焰数字/模拟量）与车位锁舵机控制。
- **网关主机**：RS485 总线调度中心，轮询从机、转发数据到串口屏与 ESP8266。
- **串口屏**：现场人机交互界面，显示状态并下发 `LOCK/GET` 指令。
- **ESP8266**：负责 WiFi 连接、MQTT 连接、网络配置持久化、云-端协议转换。
- **云端服务器**：FastAPI 后端，维护节点状态、事件日志、命令队列，提供 WebSocket 实时推送。
- **浏览器前端**：Vue/Quasar 风格的单页应用，通过 WebSocket 实时查看节点状态并远程控制。

2. 从机（Slave）主循环流程图

```
flowchart TD
    A[上电启动] --> B[初始化 RS485 UART]
    B --> C[读取硬件 UID<br/>生成节点身份]
    C --> D[发送 BOOT,node_id,START]
    D --> E[初始化外设<br/>DHT11 / 火焰DO / ADC / 舵机 PWM]
    E --> F[发送 BOOT READY 详情]
    F --> G[注册 UART 中断接收]
    G --> H[进入主循环]
    H --> G
```

```

--> I[msgq 中有<br/>完整行?] I -->|是| J[读取一行命令] J --> K[按 CSV 拆分字段] K --> L{命令前缀?} L -->|REQ,PING| M[按 node_id 计算 slot 延迟<br/>发送 ACK,node_id,PONG] L -->|REQ,GET| N[采集传感器<br/>发送 REPORT,node_id,POLL,...] L -->|SET,ADDR<br/>UID匹配| O[设置 node_id = 分配地址<br/>发送 REG,ACK,node_id,UID=...] L -->|SET,LOCK| P[设置舵机 PWM<br/>发送 ACK,node_id,LOCK,x<br/>发送 REPORT,node_id,SET,...] L -->|REQ,DISCOVER<br/>且node_id==0| Q[随机退避<br/>发送 REG,REQ,UID=...] M --> H N --> H O --> H P --> H Q --> H I -->|否| R[休眠 1ms] --> H

```

关键细节

- **总线冲突避免**：发送前检测 RS485 总线空闲时间（ RS485_IDLE_GUARD_MS=4ms ），若忙则随机退避后重试。
- **DHT 读取容错**：DHT11 在舵机动作时可能读取失败，使用上一次有效缓存值避免上报异常尖峰。
- **中断驱动接收**：UART ISR 将接收到的字符组装成行，通过 k_msgq 投递到主循环处理。

3. 网关主机（Gateway）主循环流程图

```

flowchart TD
    A[上电启动] --> B[初始化 64 个节点状态表]
    B --> C[注册三个 UART 中断<br/>RS485 / Screen / WiFi]
    C --> D[发送 GATEWAY_READY 到串口屏]
    D --> E[发送 READY,GATEWAY,1 到 ESP8266]
    E --> F[进入主循环]
    F --> G{RS485 msgq<br/>有数据?}
    G -->|是| H[process_rs485_line]
    H --> H1{帧类型?}
    H1 -->|REG,REQ| I1[ensure_addr_for_uid<br/>分配空闲地址]
    I1 --> I2["发送 SET,0,ADDR,UID,addr<br/>转发上游 ASSIGN"]
    H1 -->|REG,ACK| J1[注册 UID-地址映射<br/>标记节点 online]
    H1 -->|REPORT| K1[parse_telemetry<br/>更新节点状态]
    H1 -->|OFFLINE| K2[标记节点 offline]
    K1 --> K3[转发到 Screen + WiFi]
    K2 --> K3
    K3 --> K4[更新串口屏数值显示]
    G -->|否| L{Screen msgq<br/>有数据?}
    L -->|是| M[process_screen_line]
    M --> M1{命令类型?}
    M1 -->|LOCK/GET| N[将 Rs485Cmd 入队<br/>rs485_cmd_msgq]
    M1 -->|NETCFG/NETCLR/NETSTAT| O[直接转发到 WiFi UART]
    L -->|否| P{WiFi msgq<br/>有数据?}
    P -->|是| Q[process_wifi_line]
    Q --> Q1{命令类型?}
    Q1 -->|CMD,LOCK/GET| R[execute_gateway_command<br/>入队 Rs485Cmd]
    Q1 -->|PING,ESP| S[回复 PONG,GATEWAY]
    Q1 -->|状态行<br/>READY/NETCFG/WIFI/MQTT| T[转发到 Screen<br/>更

```

新链路状态显示] P -->|否| U{到达
RS485_TX 时刻?} U -->|是| V{rs485_cmd_msgq
有命令?} V -->|是| W[发送 LOCK/GET] V -->|否| X{到达
DISCOVERY 时刻?} X -->|是| Y[发送 REQ,0,DISCOVER] X -->|否| Z{到达
POLL 时刻?} Z -->|是| AA[find_next_registered_addr
发送 REQ,addr,GET] U -->|否| AB[遍历节点表
检查离线超时 >15s] AB --> AC[标记超时节点 OFFLINE
转发上游] AC --> AD[休眠 POLL_DELAY_MS
默认 10ms] --> F

关键细节

- **三路 UART 并发**：RS485（总线）、Screen（串口屏）、WiFi（ESP8266）各自有独立的 ISR + msgq。
- **调度优先级**：RS485 发送槽的优先级为 显式命令 > 发现探测 > 周期轮询。
- **UID 自注册**：网关通过 DISCOVER 探测未注册从机，从机以 REG,REQ 响应，网关分配地址后下发 SET,0,ADDR。
- **离线判定**：节点超过 NODE_OFFLINE_TIMEOUT_MS=15000ms 未收到任何帧即判定为离线，向上游广播 OFFLINE 事件。

4. ESP8266 WiFi 模块主循环流程图

flowchart TD
A[上电启动] --> B[初始化 Serial 115200
挂载 LittleFS]
B --> C[发送 READY,ESP,1]
C --> D{netcfg.txt 存在?}
D -->|是| E["load_config
mqtt_setup_client
发送 NETCFG,LOADED"]
D -->|否| F[发送 NETCFG,EMPTY]
E --> G[进入 loop]
F --> G
G --> H[poll_uart
读取 Serial 行]
H --> I{行类型判断}
I -->|NETCFG,...| J[process_netcfg_command
校验并保存配置
发送 NETCFG,SAVED
重置连接管道]
I -->|NETCLR| K[erase_config
断开 WiFi/MQTT
发送 NETCFG,CLEARED]
I -->|NETSTAT| L[报告 NETCFG 状态]
I -->|REPORT / OFFLINE / ACK| M[publish_raw_line
发到 MQTT 上行主题]
I -->|READY,GATEWAY,1| N[记录 gateway_ready_seen]
I -->|PONG,GATEWAY| O[记录 pong]
I -->|其他| P[报告 ERR,ESP,unknown_uart_frame]
G --> Q[handle_connectivity]
Q --> Q0{配置有效?}
Q0 -->|否| G
Q0 -->|是| R{WiFi 已连接?}
R -->|否| S["到达重试间隔?
wifi_begin_if_needed"]
R -->|是| T{MQTT 已连接?}
T -->|否| U["到达重试间隔?
mqtt_begin_if_needed"]
T -->|是| V[g_mqtt.loop]
V --> W{收到 MQTT
command 消息?}
W -->|是| X[parse_python_text_command
或 parse_python_json_command]
X --> Y[通过 Serial 发送 CMD 到网关]
Y --> G

Z[publish_python_ack
sent,serial_written] X -->|解析失败|
 Z2[publish_python_ack
failed,bad_command] G --> AA[periodic_tasks] AA --> AB{到达
 30s 状态报告周期?} AB -->|是| AC[report_link_status periodic
通过 UART 和 MQTT 同时
 报告] AB -->|否| G

关键细节

- **配置持久化**：网络配置（SSID、密码、MQTT 地址等）保存在 LittleFS 的 `/netcfg.txt` 中，掉电不丢失。
- **双格式命令支持**：云端下发的命令可以是文本格式 `CMD,id,LOCK,1,1`，也可以是 JSON 格式 `{"command_id":"...", "action":"LOCK", "node_id":1, "value":1}`。
- **链路状态聚合**：`LINK` 帧将 WiFi 状态、IP 地址、断连原因、MQTT 状态聚合为一行，同时通过 UART 发给网关并通过 MQTT 发给云端。
- **状态透传**：ESP8266 不对 `REPORT/OFFLINE/ACK` 等内容做解析，直接透传到 MQTT 上行主题。

5. 云端服务器（Cloud Backend）处理流程图

flowchart TD
 A[启动 run_embedded_server.py
或 uvicorn server:app] -->
 B[start_embedded_broker_if_needed] B --> C{外部 MQTT Broker
可连接?} C -->|是| D[使用外部 Broker] C -->|否| E[启动 amqtt
嵌入式 Broker] D --> F[start_mqtt] E --> F F -->
 G[FastAPI 启动
监听 HTTP + WebSocket] G --> H[等待请求] subgraph MQTT_Uplink
 [MQTT 上行处理] MU1[on_mqtt_message
topic = parking_lock/gateway/up/raw] -->
 MU2[handle_ingest_line] MU2 --> MU3{前缀判断} MU3 -->|REPORT| MU4["更新
NODES[node_id]
online=true
解析 T/H/FD/FA/L/UID"] MU3 -->|OFFLINE| MU5["标记
NODES[node_id]
online=false"] MU3 -->|ACK / ERR / BOOT| MU6[push_event 记录日志]
 MU4 --> MU7[push_snapshot_from_thread
WS broadcast_json] MU5 --> MU7 MU6 -->
 MU7 end subgraph MQTT_Ack [MQTT 命令回执处理] MA1[on_mqtt_message
topic =
parking_lock/gateway/command_ack] --> MA2["解析 command_id
提取 status"] MA2 -->
 MA3["更新 COMMANDS[cmd_id].status"] MA3 --> MA4[push_snapshot_from_thread
WS
broadcast_json] end subgraph HTTP_API [HTTP API 处理] HA1[POST /v1/commands] -->
 HA2[创建 CommandItem
生成 UUID] HA2 --> HA3[publish_command
发送 CMD 到
MQTT] HA3 --> HA4[push_event 记录命令创建] HA4 --> HA5[WS broadcast_json] HA6[POST

```

/v1/ingest/line] --> HA7[handle_ingest_line] HA7 --> HA8[WS broadcast_json] HA9[POST /v1/
ingest/batch] --> HA10[批量 handle_ingest_line] HA10 --> HA11[WS broadcast_json]
HA12[GET /v1/nodes] --> HA13[返回 NODES 列表] HA13[GET /v1/events] --> HA14[返回
EVENTS 最近N条] HA15["POST /v1/commands/{id}/ack"] --> HA16["更新命令状态为 done/
failed"] HA16 --> HA17[WS broadcast_json] end subgraph WS [WebSocket 连接管理] WS1[客
户端连接 /ws] --> WS2[ws_manager.connect] WS2 --> WS3[发送当前 make_snapshot] WS3 --
> WS4[保持连接等待广播] WS5[broadcast_json] --> WS6[向所有存活客户端<br/>发送
snapshot JSON] end

```

关键细节

- **内存存储**：节点状态 `NODES`、事件队列 `EVENTS`、命令表 `COMMANDS` 均为内存结构，重启后清空。
- **实时广播**：任何导致状态变化的事件（REPORT、OFFLINE、命令创建、命令 ACK）都会触发 WebSocket 广播，前端无需轮询。
- **双通道接入**：网关数据既可以通过 `serial_mqtt_bridge.py` 经 MQTT 进入，也可以通过 `serial_http_bridge.py` 直接调用 HTTP `/v1/ingest/line` 进入。
- **命令生命周期**：`pending` -> `sent` (publish 成功) -> `done/failed` (收到网关 ACK)。

6. 前端 Web UI 交互流程图

```

flowchart TD
    A[页面加载] --> B[initTheme<br/>读取 localStorage 或系统偏好]
    B --> C[refreshAll<br/>并行请求 /v1/nodes /v1/events /health]
    C --> D[renderNodes<br/>渲染在线节点卡片]
    D --> E[renderEvents<br/>渲染最近事件列表]
    E --> F[connectWs<br/>连接 WebSocket /ws]
    F --> G{收到 ws.onmessage}
    G -->|type=snapshot| H[applySnapshot]
    H --> I[renderNodes<br/>更新显示]
    I --> J[renderEvents<br/>更新事件]
    J --> K[checkPendingLocks]
    K --> L{pendingLocks 中存在未达成预期状态的节点?}
    L -->|是且收到自然回包| M[retriesLeft--<br/>自动重发 POST /v1/commands]
    L -->|重试次数耗尽| N[提示失败并清除 pending]
    N --> H
    H --> O[maybeAlarm]
    O --> P{存在火焰告警? flame_digital==0 或 risk>=70%}
    P -->|是且用户启用了声音| Q[beepOnce 播放蜂鸣]
    Q --> R[用户点击锁定按钮]
    R --> S[pendingLocks.set<br/>expectedValue=1<br/>retriesLeft=20]
    S --> T[sendCommand<br/>POST /v1/commands LOCK]
    T --> U[状态栏显示 命令已入队]
    U --> V[用户点

```

击解锁按钮] --> W[pendingLocks.set
expectedValue=0] W -->
X[sendCommand
POST /v1/commands LOCK value=0] Y[用户点击刷新按钮] -->
Z[refreshAll] AA[用户点击启用警示音] --> AB[ensureAudio
alarmEnabled=true]

关键细节

- **自动重试机制**：前端在点击锁定/解锁后，会进入 `pendingLocks` 状态。若 WebSocket 推送的节点快照中锁状态未变为预期值，且该节点的 `updated_at` 有变化（说明收到了新的自然回包），则自动重试，最多 20 次。
- **ACK 过滤**：网关的命令执行回执（ACK）不会更新节点的 `updated_at`，因此不会误触发自动重试。
- **主题切换**：支持亮/暗主题，优先使用用户手动选择，否则跟随系统偏好。
- **30 秒兜底轮询**：即使 WebSocket 断开，每 30 秒也会通过 HTTP 刷新一次，保证数据不会长期停滞。

7. 设备注册与轮询时序图

sequenceDiagram participant S as 从机
node_id=0 participant G as 网关主机 participant SCR as 串口屏 participant ESP as ESP8266 participant Cloud as 云端服务器 Note over S: 上电后 node_id 为 0,
等待发现广播 loop 每 1 秒 G->>G: 发送 REQ,0,DISCOVER end S->>S: 随机退避 (2~14ms) S->>G: REG,REQ,UID=12AB34CD G->>G: 分配首个空闲地址 addr=1 G->>S: SET,0,ADDR,12AB34CD,1 G->>SCR: 转发 REG,ASSIGN,1,UID=12AB34CD G->>ESP: 转发 REG,ASSIGN,1,UID=12AB34CD ESP->>Cloud: MQTT publish 上行 S->>S: 验证 UID 匹配
设置 node_id=1 S->>G: REG,ACK,1,UID=12AB34CD G->>G: 注册 UID 到地址 1
标记节点 1 online G->>SCR: 转发 REG,ACK G->>ESP: 转发 REG,ACK ESP->>Cloud: MQTT publish 上行 Note over G,S: 注册完成, 进入周期轮询 loop 每 200ms 轮询一个注册节点 G->>S: REQ,1,GET S->>S: 读取 DHT11 / 火焰 / 锁状态 S->>G: REPORT,1,POLL,T=26.00,H=55.00,FD=1,FA=3289,L=0,UID=... G->>G: 更新节点 1 状态缓存 G->>SCR: 转发 REPORT G->>ESP: 转发 REPORT ESP->>Cloud: MQTT publish parking_lock/gateway/up/raw Cloud->>Cloud: 更新内存节点状态 Cloud->>BROWSER: WebSocket broadcast snapshot end

8. 云到端控制命令完整时序图

sequenceDiagram participant UI as 浏览器前端 participant Cloud as FastAPI 云端 participant MQTT as MQTT Broker participant ESP as ESP8266 participant G as 网关主机 participant S as 从机 participant SCR as 串口屏 Note over UI,S: 场景：用户远程锁定节点 1 UI->>Cloud: POST /v1/commands{node_id:1, action:"LOCK", value:1} Cloud->>Cloud: 创建 CommandItem生成 command_id = uuid4 Cloud->>MQTT: publish parking_lock/cloud/commandCMD,uuid,LOCK,1,1 Cloud->>UI: HTTP 返回 command 详情 MQTT->>ESP: 推送 command 消息 ESP->>ESP: parse_python_text_command ESP->>G: 通过 UART 发送CMD,uuid,LOCK,1,1 ESP->>MQTT: publish parking_lock/gateway/command_ackACK,uuid,sent,serial_written G->>G: process_wifi_line验证节点 1 已注册 G->>G: 将 LOCK 命令入队 rs485_cmd_msgq G->>MQTT: 通过 WiFi UART 转发 ACKACK,uuid,sent,rs485_written(ESP 再 publish 到 MQTT) G->>S: RS485 发送SET,1,LOCK,1 S->>S: 驱动舵机到锁定位置 S->>G: ACK,1,LOCK,1 S->>G: REPORT,1,SET,T=...,H=...,L=1,... G->>SCR: 转发 REPORT 与 ACK更新屏幕显示 G->>ESP: 转发 REPORT 与 ACK ESP->>MQTT: publish 上行 parking_lock/gateway/up/raw MQTT->>Cloud: 接收 REPORT / ACK Cloud->>Cloud: handle_ingest_line更新节点 1 锁状态为 1更新 command 状态 Cloud->>UI: WebSocket broadcast snapshot UI->>UI: 渲染节点 1 为 LOCKED清除 pendingLocks

9. 离线检测与上报流程图

flowchart TD A[网关主循环每 10ms 执行一次] --> B[遍历所有 64 个节点状态] B --> C{节点 online=true 且 now - lastSeen > 15000ms?} C -->|是| D[标记 online=false] D --> E[构造 OFFLINE,node_id 帧] E --> F[forward_upstream] F --> G[发送到串口屏] G --> H[发送到 ESP8266] H --> I[ESP publish 到 MQTT] I --> J[云端 handle_ingest_line] J --> K[标记节点 offline] K --> L[push_event] L --> M[WS broadcast snapshot] M --> N[前端显示节点离线] C -->|否| O[继续检查下一个节点]

附录：协议速查表

RS485 从机-网关协议（ASCII, \r\n 结尾）

方向	帧格式	说明
网关->从机	REQ,<node_id>,PING	心跳探测
网关->从机	REQ,<node_id>,GET	请求上报传感器数据
网关->从机	SET,<node_id>,LOCK,<0 1>	控制车位锁
网关->从机	REQ,0,DISCOVER	广播发现未注册从机
网关->从机	SET,0,ADDR,<UID>,<addr>	给指定 UID 分配地址
从机->网关	ACK,<node_id>,PONG	PING 响应
从机->网关	REPORT,<node_id>,<reason>,T=...,H=...,FD=...,FA=...,L=...	状态上报
从机->网关	REG,REQ,UID=...	请求注册
从机->网关	REG,ACK,<addr>,UID=...	注册确认

网关-ESP8266 UART 协议

方向	帧格式	说明
网关->ESP	READY,GATEWAY,1	网关启动就绪

方向	帧格式	说明
网关->ESP	REPORT,... / OFFLINE,... / REG,...	透传上行数据
网关->ESP	NETCFG,<ssid>,...,<mqtt_pass>	网络配置
网关->ESP	NETCLR	清除配置
网关->ESP	ACK,<cmd_id>,<status>,<code>	命令执行回执
ESP->网关	READY,ESP,1	ESP 启动就绪
ESP->网关	CMD,<cmd_id>,LOCK,<node_id>,<0 1>	云端下发命令
ESP->网关	CMD,<cmd_id>,GET,<node_id>	云端请求轮询
ESP->网关	LINK,... / WIFI,... / MQTT,...	链路状态报告

MQTT 主题

主题	方向	说明
parking_lock/gateway/up/raw	网关 -> 云端	原始上行数据
parking_lock/cloud/command	云端 -> 网关	下行控制命令
parking_lock/gateway/command_ack	网关 -> 云端	命令执行回执